Attack Simulation Report — SQL Injection (DVWA via Burp Suite)

1. Attack Overview

This report documents a simulated SQL injection attack against DVWA (Damn Vulnerable Web Application), hosted on the Metasploitable2 target in the Wazuh homelab, using Burp Suite Community Edition as the attack platform.

Field	Detail
Attack type	SQL Injection (authentication/filter bypass, data exfiltration)
Tool	Burp Suite Community Edition (Proxy + Repeater)
Attacker	Kali Linux (10.0.2.15)
Target	DVWA on Metasploitable2 Apache/MySQL (10.0.2.3), SQL Injection module, security level: Low
Injection point	`id` GET parameter, `/dvwa/vulnerabilities/sqli/`
MITRE ATT&CK	T1190 (Exploit Public-Facing Application)
Detection rule	Wazuh custom rule 100018 (added during this exercise)

Objective: intercept and manipulate DVWA's SQL Injection module through Burp Suite to confirm and exploit the vulnerability, extend Wazuh's logging to make raw web traffic browsable (not just alerts), and build a custom detection rule validated against the live attack.

2. Target and Tool Setup

Burp Suite's built-in Chromium browser was used to route all traffic through the Proxy, avoiding manual browser proxy configuration. DVWA was accessed at `http://10.0.2.3/dvwa/`, authenticated with the default `admin:password`, and the DVWA Security level was set to **Low** to expose the vulnerability without input filtering.

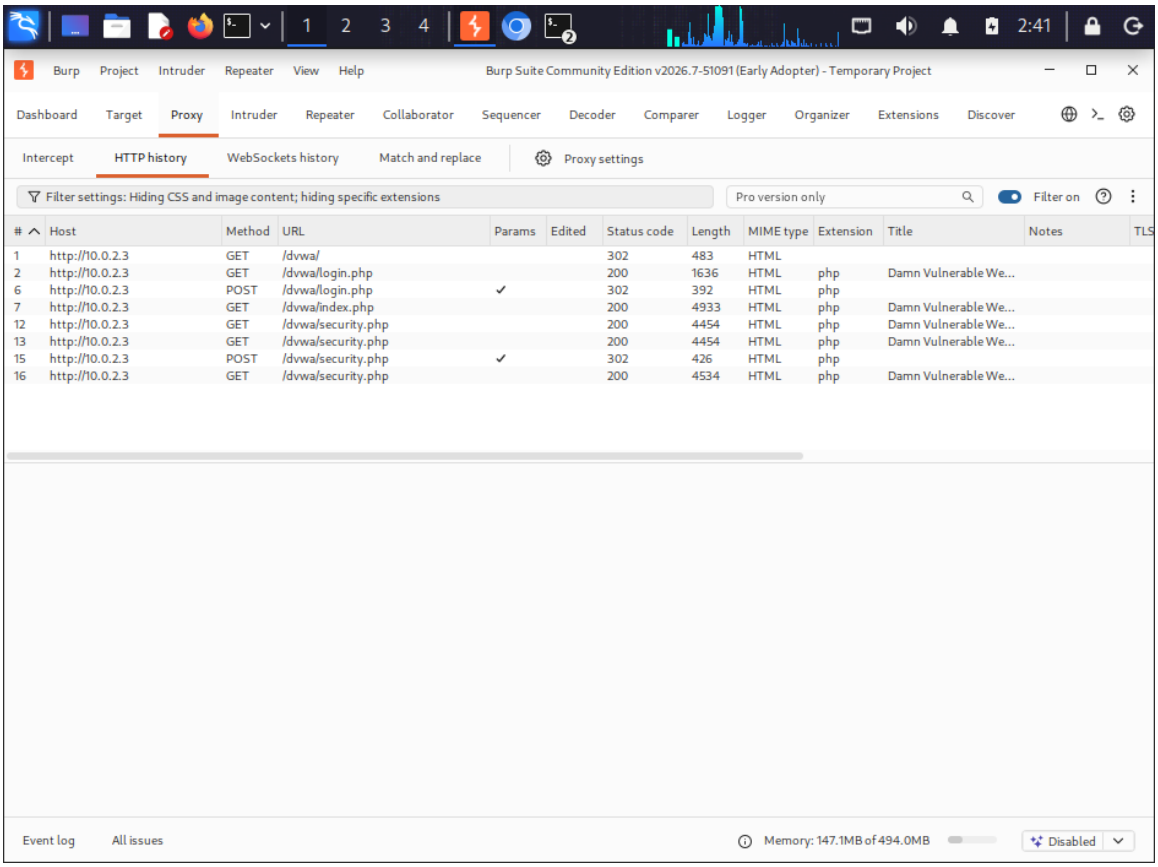


Figure 1: Burp Suite Proxy → HTTP History, confirming traffic from the DVWA session — login POST, security.php changes, and page navigation — flowing cleanly through the proxy before any attack traffic was sent.

Apache Log Pipeline Setup

For any of this traffic to reach Wazuh at all, Apache's logs first had to be forwarded off Metasploitable2 — a prerequisite completed before this attack, covered here for completeness. The standard approach (`rsyslog`'s `imfile` module tailing `access.log`/`error.log`) turned out not to be viable: debugging via `rsyslogd -d -n` showed this build's `imfile.so` module simply doesn't exist on disk (`cannot open shared object file`), and no `rsyslog-imfile` package was available in the repos either.

The workaround used Apache's own built-in piped-logging feature instead — no missing module required. In `/etc/apache2/sites-enabled/000-default`, the `CustomLog` and `ErrorLog` directives were changed to pipe each log line directly into `logger`, a standard syslog client every Linux box ships with:

```
CustomLog "|/usr/bin/logger -p local6.info -t apache-access" combined
ErrorLog "|/usr/bin/logger -p local6.info -t apache-error"
```

This feeds straight into the lab's existing `*.* @10.0.2.4:514` rsyslog forwarding rule — the same mechanism already carrying vsftpd, SSH, and sudo logs to Wazuh — so no changes were needed on the Wazuh side for basic log receipt. The config was validated with `apache2ctl configtest` before restarting Apache, then confirmed end-to-end: a test `curl` request showed up tagged `apache-access` in

Metasploitable2's own `/var/log/syslog` within seconds, and the same request was independently confirmed in Wazuh's `archives.log` via `grep` on the Wazuh VM.

3. Attack Execution and Diagnostics

A baseline request (`id=1`) was captured from Burp's HTTP History and sent to **Repeater**, confirming a clean response: `ID: 1 / First name: admin / Surname: admin`. Initial attempts to inject `1' OR '1'='1` directly through Repeater's Raw editor returned blank output with no SQL error — narrowing down the cause required ruling out several possibilities in turn: a stray literal space breaking the HTTP request line, `magic_quotes_gpc` auto-escaping input (checked directly via `phpinfo.php` and found **Off**), and an expired DVWA session silently redirecting to login.

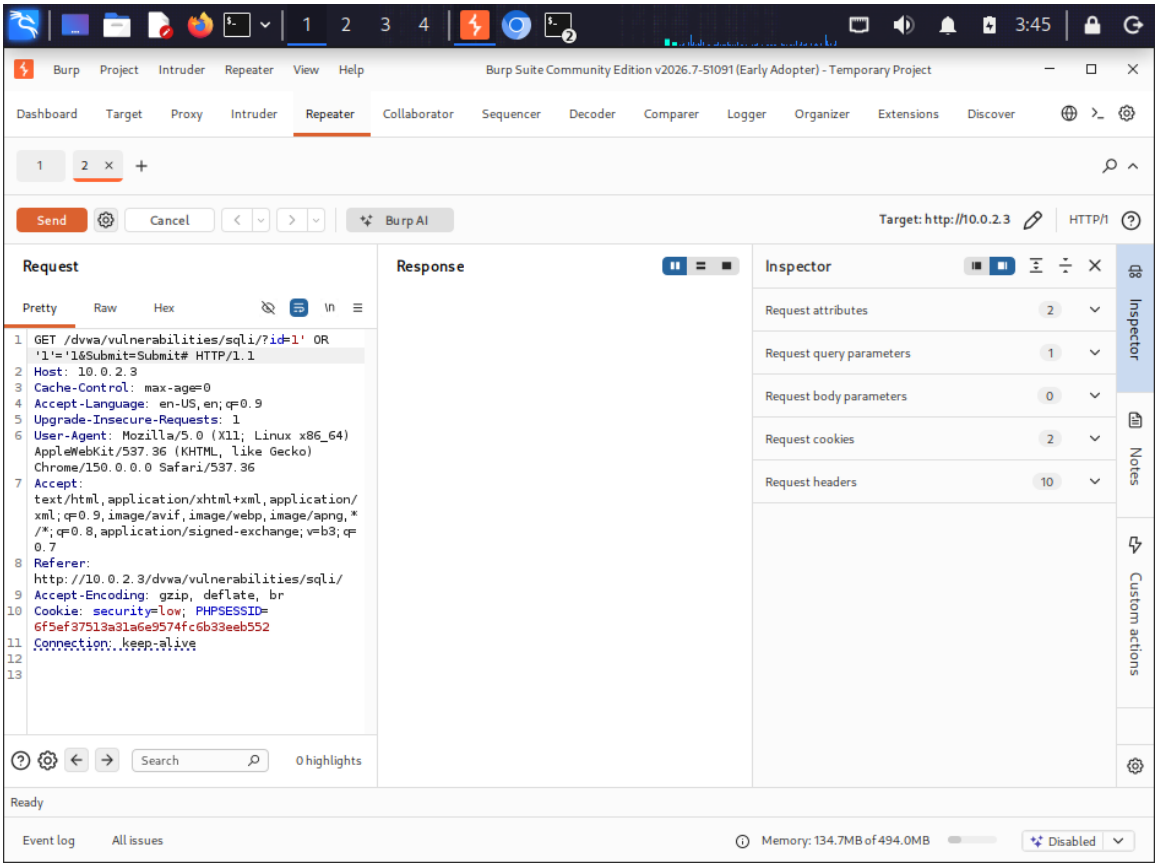


Figure 2: Burp Repeater, Raw view — the injection payload `id=1%27%20OR%20%271%27%3D%271` encoded explicitly to avoid the literal-space parsing issue that caused earlier attempts to fail silently.

The injection point was first confirmed reachable with a minimal diagnostic payload — a single trailing quote (`id=1'`) — which returned a genuine MySQL syntax error:

```
You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''1''' at line 1
```

This confirmed user input reaches the SQL query unescaped (classic error-based SQL injection), and revealed the query wraps `id` in single quotes — explaining why the earlier OR-bypass payload needed proper quote placement rather than assuming a numeric, unquoted column.

4. Successful Exploitation — Data Exfiltration

With the quote placement corrected and re-authenticated against a fresh session, the OR-based bypass payload was sent:

```
GET /dvwa/vulnerabilities/sqli/?id=1%27%20OR%20%271%27%3D%271&Submit=Submit HTTP/1.1
```

This broke the query's `WHERE` clause entirely, returning **every row in the users table** instead of the single requested ID — confirmed data exfiltration, not just a syntax-error proof of concept:

```
ID: 1' OR '1'='1   First name: admin    Surname: admin
ID: 1' OR '1'='1   First name: Gordon   Surname: Brown
ID: 1' OR '1'='1   First name: Hack     Surname: Me
ID: 1' OR '1'='1   First name: Pablo    Surname: Picasso
ID: 1' OR '1'='1   First name: Bob      Surname: Smith
```

All 5 DVWA user records were dumped in a single request — the strongest effect-based evidence for this attack, equivalent to credential exfiltration in a real deployment.

5. Raw Log Pipeline — Extending Dashboard Visibility

By default, Wazuh's Discover view only surfaces events that matched a rule (`wazuh-alerts-*`); unmatched raw traffic sits only in `archives.log` on disk, verifiable by terminal but not browsable in the dashboard. Since Apache web traffic had no existing rule, this attack's requests were initially invisible in Discover even with full confirmation via `grep` on the Wazuh VM.

To close this gap, Wazuh's archives indexing was enabled: `logall_json` was added to `ossec.conf`, and Filebeat's `archives.enabled` setting (found directly in `filebeat.yml` on this all-in-one OVA install, rather than a separate module file) was switched to `true`. A new `wazuh-archives-*` index pattern was then created in the dashboard.

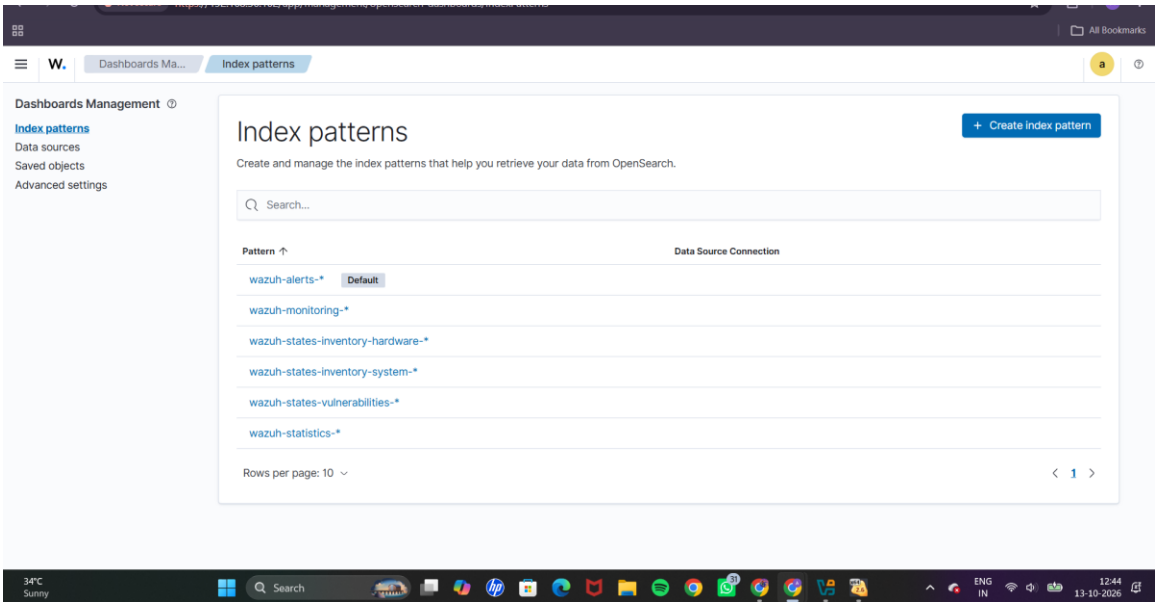


Figure 3: Dashboards Management → Index patterns — creating the new `wazuh-archives-*` pattern to surface raw, unalerted log data.

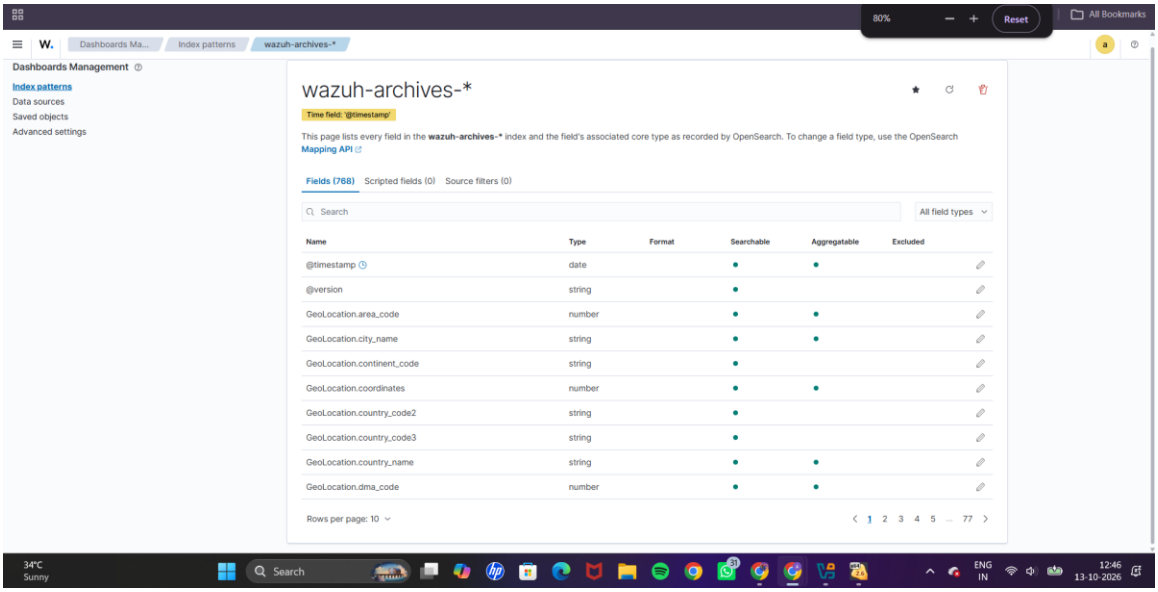


Figure 4: The `wazuh-archives-*` index pattern confirmed live, with 768 mapped fields — proof the archive data is structured and queryable, not just raw text.

With the pipeline live, the SQL injection request became fully browsable in Discover — decoded into structured fields (`data.url`, `data.srcip`, `decoder.name: web-accesslog`) rather than a raw blob:

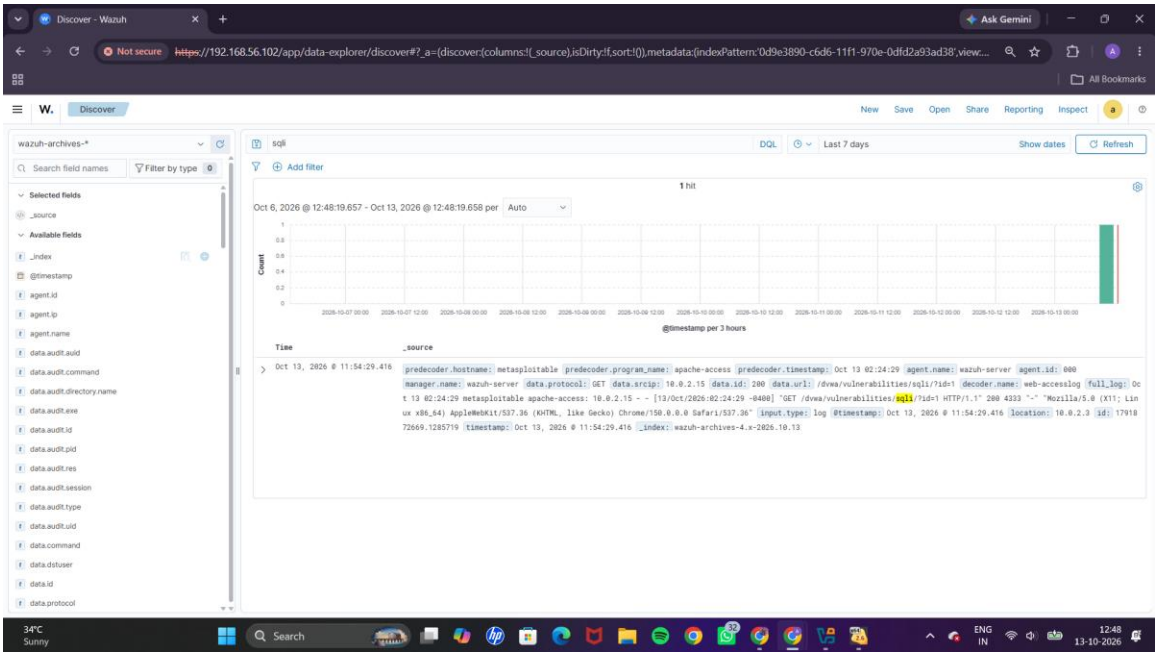


Figure 5: Discover, index pattern `wazuh-archives-*`, query `sqli` — the injection request now appears as a structured, searchable document, independent of whether any alert rule matches it.

6. Remediation — Custom Rule 100018

The injection request's raw log line was tested via the dashboard's Ruleset Test tool against the live ruleset:

```
Oct 13 03:42:02 metasploitable apache-access: 10.0.2.15 - - [13/Oct/2026:03:42:01 -
0400] "GET /dvwa/vulnerabilities/sqli/?id=1%27%20OR%20%271%27%3D%271&Submit=Submit
HTTP/1.1" 200 4660 ...
```

Only the generic built-in rule **31100** ("Access log messages grouped", level 0, groups `web,accesslog`) matched — confirming no SQLi-specific detection existed yet, the same situation FTP brute force started in before rule 100017.

A new rule was added to `local_rules.xml`, built on top of 31100 and matching known SQL injection syntax patterns against the decoder's extracted `url` field:

```
<rule id="100018" level="10">
  <if_sid>31100</if_sid>
  <url type="pcre2">(?i)(%27|'|union\s+select|or\s+1=1|--|;--|drop\s+table)</url>
  <description>Web Attack: possible SQL Injection attempt detected</description>
  <group>web,attack,sql_injection,</group>
  <mitre>
    <id>T1190</id>
  </mitre>
</rule>
```

```
28      <description>Multiple failed SSH authentication attempts from same source (possible brute force) - 5 failures in 2 min
29      <group>authentication_failures,</group>
30    </rule>
31
32    <rule id="100014" level="12">
33      <if_matched_sid>5760</if_matched_sid>
34      <if_sid>5715</if_sid>
35      <same_source_ip />
36      <description>Successful SSH login after multiple failures from same IP - possible account compromise</description>
37      <group>authentication_success,attack,</group>
38    </rule>
39
40  <rule id="100017" level="10" frequency="5" timeframe="120">
41    <if_matched_sid>5503</if_matched_sid>
42    <program_name>vsftpd</program_name>
43    <same_source_ip />
44    <description>FTP brute force: 5 failed logins in 2 minutes</description>
45  </rule>
46
47  <rule id="100018" level="10">
48    <if_sid>31100</if_sid>
49    <url type="pcre2">(?i)(%27|'|union\s+select|or\s+1=1|--|;--|drop\s+table)</url>
50    <description>Web Attack: possible SQL Injection attempt detected</description>
51    <group>web,attack,sql_injection,</group>
52    <mitre>
53      <id>T1190</id>
54    </mitre>
55  </rule>
56
57  </group>
```

Figure 6: `local_rules.xml`, rule 100018 — matches encoded/literal quotes, `UNION SELECT`, `OR 1=1`, and SQL comment markers against the decoded request URL, scoped to web-access logs via `if_sid: 31100`.

Re-testing the same raw log line through Ruleset Test confirmed the rule fired correctly: `id: 100018`, `level: 10`, `mitre.id: T1190`, `Alert to be generated` — validated before any new traffic was generated.

7. Validation — Live Alert Confirmation

The injection payload was re-sent through Burp, generating fresh traffic against the live ruleset. Checking Threat Hunting → Events with `rule.level >= 10` confirmed rule 100018 firing as a classified, high-severity alert:

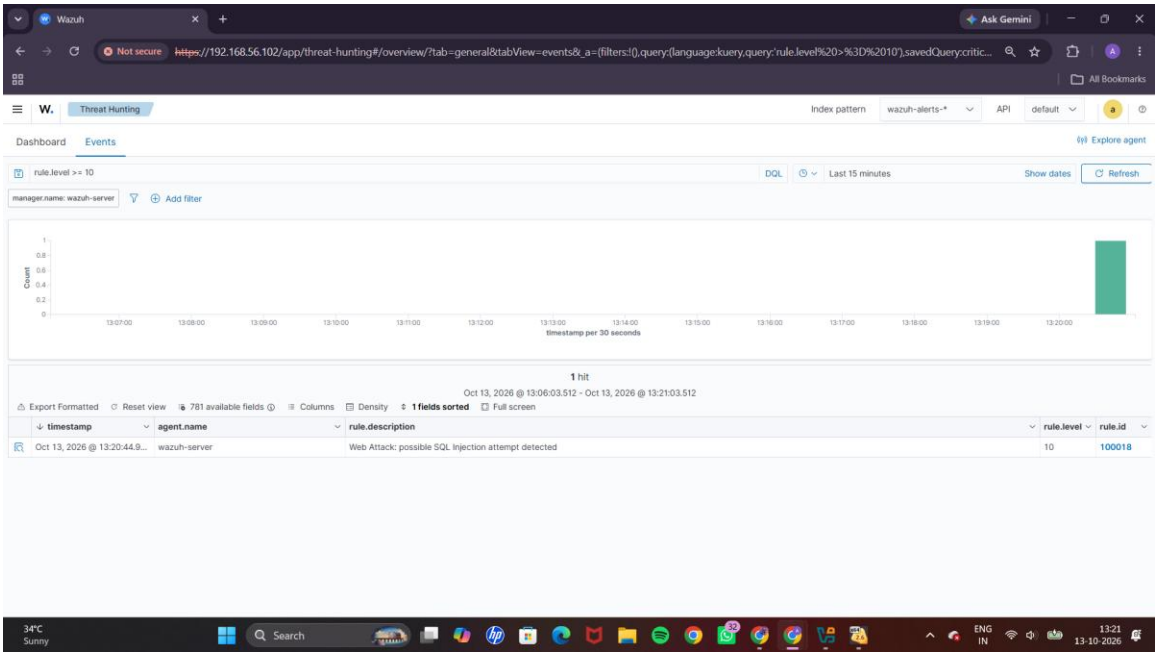


Figure 7: Threat Hunting → Events — rule 100018 surfaced at level 10, alongside the dashboard's other high-severity detections, confirming the SQLi attack is now classified rather than sitting only in raw logs.

Expanding the alert's full document details confirms the complete evidentiary chain — source IP, decoded URL with the injection payload intact, the matching decoder, and the rule's MITRE mapping, all in one record:

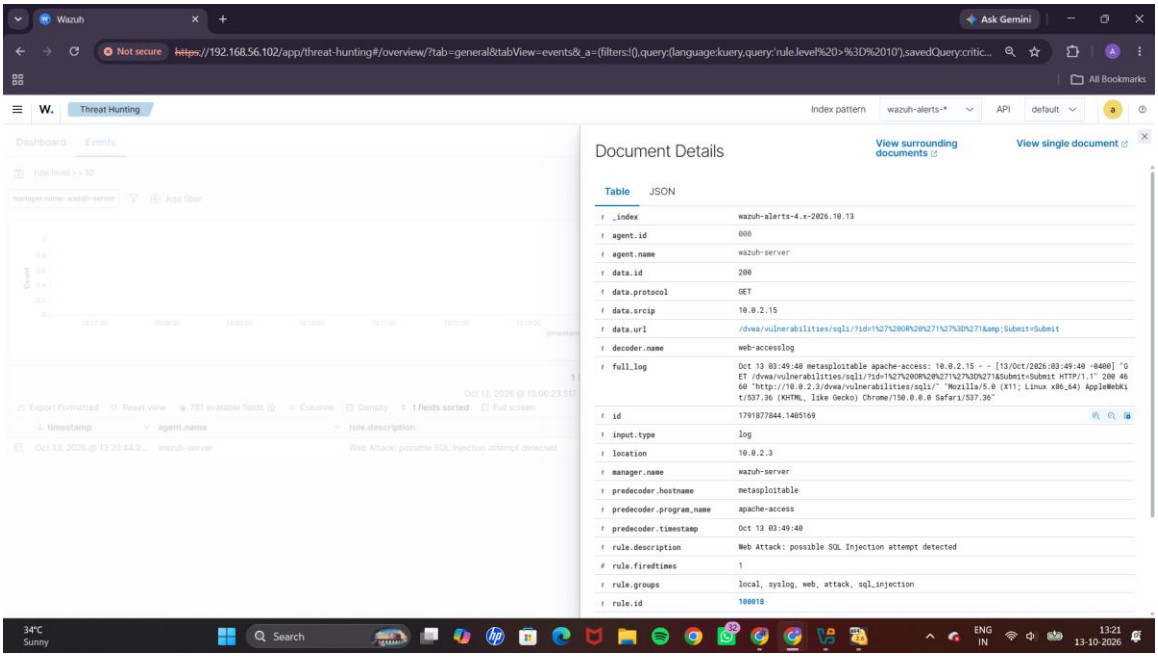


Figure 8: Document Details for the alert — `data.url: /dvwa/vulnerabilities/sqli/?id=1%27%20OR%20%271%27%3D%271&Submit=Submit`, `data.srcip: 10.0.2.15`, `rule.id: 100018`, `rule.groups: web, attack, sql_injection` — a complete, self-contained record of the detected attack.

8. Findings & Conclusion

Scenario	Result
Error-based injection probe (`id=1'`)	Confirmed reachable, unescaped input — MySQL syntax error returned
OR-bypass (`id=1' OR '1'='1`)	All 5 DVWA user records exfiltrated in one request
Pre-remediation alert check	No alert generated — only generic rule 31100 (level 0) matched
Post-remediation alert check	Rule 100018 fired correctly at level 10, with full MITRE T1190 mapping

This exercise extended the homelab's detection coverage into web application attacks for the first time, closing a gap parallel to FTP's: no SQLi-specific rule existed despite SSH, sudo, and FTP brute force already being covered. Rule 100018 was built the same way as 100017 — confirm the base rule via Ruleset Test first, then layer a custom match on top — but required pattern matching against decoded request content rather than a frequency-based approach, since SQL injection is a single-request attack rather than a repeated-failure pattern.

Beyond the detection rule itself, this exercise also closed an infrastructure gap: Wazuh's dashboard by default only surfaces rule-matched events, leaving all other web traffic invisible to analysts browsing Discover. Enabling the `wazuh-archives-*` index gives permanent visibility into every request reaching

Apache — attack traffic and benign traffic alike — independent of whether a rule exists to classify it yet. This is a reusable capability for any future web-based attack simulation (XSS, command injection, CSRF) run against this lab.